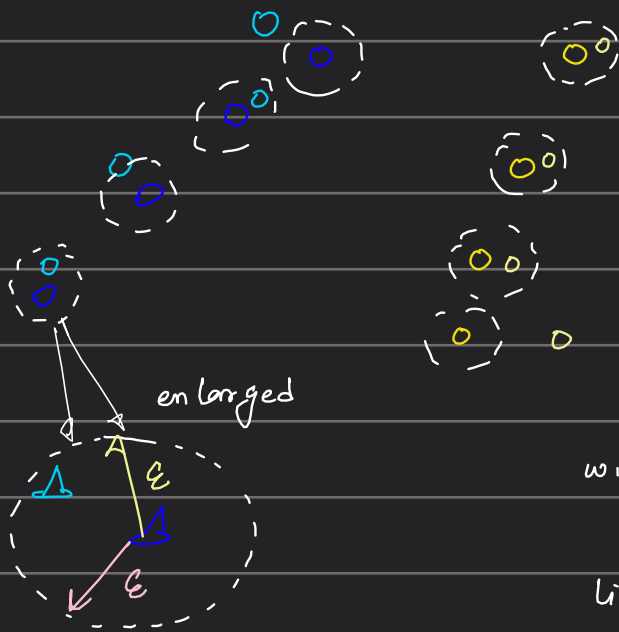


SLAM-Checkpoint-3

**Nearest Neighbors
(python)**

Data Association — Nearest Neighbours

- ↳ finding the match for a cone (true coord & coord identified by perception)
- ↳ for each iteration getting making the coord of predicted cones better thus improving the map at every iteration



— blue cones estimated by perception

— yellow cones estimated by perception

— true position of blue cones

— true position of yellow cones

ϵ : draw a circle of radius ϵ (error bound) with the centre at a true cone such that the perception cone which lies inside this circle will be considered the predicted pose of the cone at the centre itself.

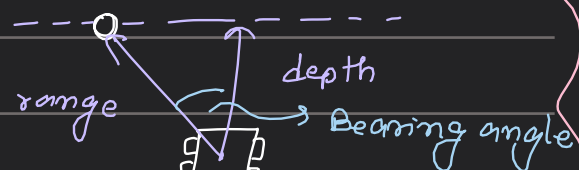
Given: ① The car's state at time t as $[x, y, \psi]^T$

② Set of pose of cones predicted by perception

$[(r_1, \theta_1), (r_2, \theta_2) \dots (r_m, \theta_m)]$
range bearing angle

Note:

$$\text{range} \times \cos(\text{bearing}) = \text{depth}$$



③ True pose of the cones (map)

$[(x_1, y_1), (x_2, y_2), \dots (x_m, y_m)]$

```
def compute_nearest_neighbors(perceived_cones, real_cones, threshold):
```

```
    """
```

```
    Returns an array of indices:
```

```
    - index of nearest real cone
```

```
    - OR -11 if no valid match
```

```
    """
```

```
    matches = []
```

```
    for p in perceived_cones:
```

```
        distances = []
```

```
        for r in real_cones:
```

```
            d = np.sqrt((p[0] - r[0])**2 + (p[1] - r[1])**2)
```

```
            distances.append(d)
```

```
        distances = np.array(distances)
```

```
        min_index = np.argmin(distances)
```

```
        min_distance = distances[min_index]
```

```
        if min_distance <= threshold:
```

```
            matches.append(min_index)
```

```
        else:
```

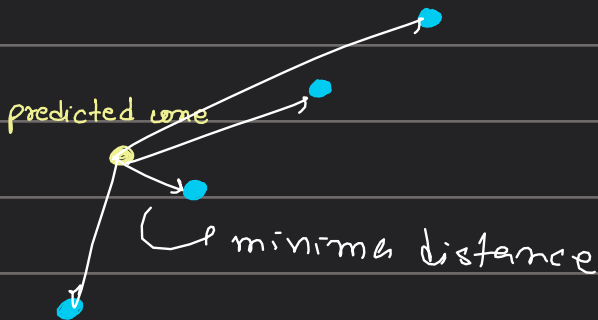
```
            matches.append(-1)
```

```
    return np.array(matches)
```

→ computing the distance b/w the particular predicted cone & all the true cones

→ finding that particular true cone which is at the minimum distance from the predicted cone

→ comparison



* here I have assumed that the minimum distance b/w the true cone

& the predicted cone is unique i.e.

a true cone is at a particular minimum distance from one particular predicted cone only